

Caronascar

Marketplace de Caronas Intermunicipais

Aluno: **Breno de Oliveira Brandão**

1. Descrição do Domínio

O Caronascar é um marketplace de caronas intermunicipais que conecta motoristas que planejam fazer uma viagem a passageiros que precisam ir ao mesmo destino. O Caronascar segue o modelo de publicação antecipada de viagens: o motorista anuncia a rota, a data, o horário de partida e o número de vagas disponíveis, os passageiros buscam viagens compatíveis com seu itinerário e solicitam uma vaga. O motorista então aceita ou recusa cada solicitação.

Esse modelo é inspirado em plataformas como BlaBlaCar e é especialmente adequado para viagens entre cidades. A comunicação entre os componentes é orientada a eventos (EDA): cada ação relevante, publicação de viagem, solicitação de vaga, aceite, recusa, cancelamento, gera um evento publicado no middleware de mensagens (MOM), garantindo notificações em tempo real.

2. Justificativa

- Distinção clara entre perfis: o motorista é o prestador, ele detém o recurso (viagem/vagas) e decide quem embarca. O passageiro é o cliente, ele busca e solicita acesso a esse recurso.
- Riqueza de eventos de domínio: a separação entre a viagem (entidade do motorista) e a solicitação de vaga (entidade do passageiro) gera múltiplos eventos naturais e bem definidos.

3. Perfis de Usuário e principais funcionalidades

3.1 Motorista (Prestador de Serviço)

Condutor cadastrado que planeja fazer uma viagem e deseja compartilhá-la. Suas ações principais são:

- Publicar uma viagem: informar origem, destino, data/hora de partida, número de vagas e preço sugerido por vaga.
- Gerenciar solicitações: receber notificações assíncronas de passageiros interessados, aceitar ou recusar cada solicitação.
- Atualizar o status da viagem: marcar como em andamento e concluída.
- Cancelar uma viagem publicada.

3.2 Passageiro (Cliente)

Pessoa que precisa viajar e busca uma carona disponível. Suas ações principais são:

- Buscar viagens disponíveis filtradas por origem, destino e data.
- Solicitar uma vaga em uma viagem de interesse.
- Receber notificação quando o motorista aceitar ou recusar sua solicitação.
- Cancelar sua solicitação de vaga enquanto ainda está pendente ou aceita.

4. Visão Geral da Arquitetura

O sistema é composto pelos seguintes componentes, detalhados no Diagrama de Arquitetura:

- **App Motorista (Flutter/Dart):** gerencia viagens, recebe notificações de solicitações de vaga e atualiza status via REST e WebSocket.
- **App Passageiro (Flutter/Dart):** busca viagens, solicita vagas e acompanha notificações de aceite ou recusa via REST e WebSocket.
- **API REST (Node.js + Express):** expõe os endpoints de viagens e solicitações de vaga, gerencia as conexões WebSocket e é responsável pela lógica de negócio e persistência.
- **Worker (BullMQ):** processo dedicado que consome jobs do Redis e publica os resultados no canal Pub/Sub para que a API os entregue aos apps via WebSocket.
- **Redis:** serve como fila persistente do BullMQ entre API e Worker, e como broker Pub/Sub para comunicação desacoplada entre os dois processos.
- **PostgreSQL:** armazena as entidades do domínio nas tabelas users, trips e seat_requests.

4.1 Diagrama da Arquitetura

